

SOFTWARE CONFIGURATION
MANAGEMENT

Edward H. Bersoff
Vilas D. Henderson
Stan G. Siegel

CTEC, Inc.
7777 Leesburg Pike
Falls Church, Virginia
22043

1.0 INTRODUCTION

Does any of the following sound familiar or conjure up painful memories:

"Sir, before I hand in my resignation, I would like you to hear my side of the story as to why the software we promised the customer two years ago has not been delivered."

"Bob Bitbuster--the whiz programmer who left the company last week--is the only guy who knows what this computer program is supposed to do".

"Does anybody know why we have the same subroutine called by three different names?"

This paper is about discipline. It is about discipline that managers should apply to software development. Why is such discipline needed? Quite simply because the software industry has traditionally behaved in an undisciplined manner--doing its own thing. The products that the industry has turned out have typically

Contained other than what was expected (usually less, rather than more);

Been delivered much later than scheduled;

Cost more than anticipated;

Been poorly documented; and

Been poorly designed.

If you have been involved in any of the situations quoted above, then this paper may be of some help. In short, if you are now, or intend to be, a software seller or buyer, then you should benefit from an understanding of Software Configuration Management. Lest you think that you are not now, or ever will be, a software seller or buyer--keep in mind that the recent technology explosion in electronic component miniaturization has placed the era of personalized computing at hand. In that context, nearly everyone may be considered a potential seller or buyer of software.

This paper is about the discipline called Software Configuration Management (SCM). The

objective of SCM is to assist the software seller in achieving product integrity and to assist the software buyer in obtaining a product that has integrity.

Configuration Management (CM) is a management discipline traditionally applied to hardware development whose application in conjunction with other disciplines leads to orderly and structured system development. CM is generally concerned with the consistent labeling, tracking, and change control of the hardware elements of a system. Software Configuration Management (SCM) is CM tailored to a system, or portion of a system, predominantly comprised of software. The purpose of this paper is to present an overview of the elements that make up the SCM discipline.

Why is there a need for a separate discipline pertaining specifically to configuration management of software? As Figure 1-1 suggests, the development of systems consisting of both hardware and software has indeed been accomplished under the CM umbrella. However, this figure also indicates that published CM guidance is generally inconsistent in its treatment of hardware versus its treatment of software. The orientation of published guidance has been toward a treatment of the hardware subsystem in microscopic detail and, at the other extreme, a treatment of the software subsystem as a single entity.^{a-j} This superficial treatment of the software component has suppressed its visibility during overall system evolution. This lack of visibility has typically resulted in a corresponding lack of management control (How can you manage what you cannot see?). This lack of management control has, in turn, resulted in the problems referred to in the opening paragraphs of this paper.

This paper, therefore, addresses a current void in the area of software development management practice. SCM does not differ substantially from CM in terms of objectives, need and functions to be performed. CM of hardware-oriented systems is generally well understood and effectively practiced. However, attempts at effective implementation of SCM have often failed because particulars of this implementation do not follow by direct analogy from hardware CM practices. SCM is an immature discipline when contrasted with hardware CM. We at-

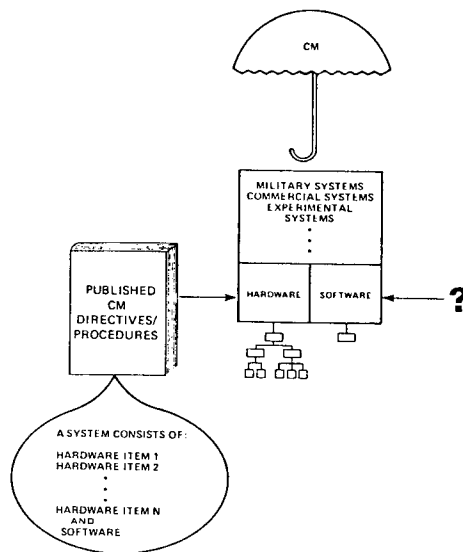


FIGURE 1-1. THE NEED FOR SOFTWARE CONFIGURATION MANAGEMENT METHODOLOGY

tempt to bring the SCM discipline a further step toward maturity by highlighting the elements of a discipline that software development managers can apply to their projects.

2.0 CONCEPT OF SOFTWARE

To understand the elements that comprise the SCM discipline, it is necessary to formulate a concept of software. The purpose of this section is to provide this formulation.

Software is information that is

- (1) Structured with logical and functional properties,
- (2) Created and maintained in various forms and representations during its life cycle, and
- (3) Tailored for machine¹ processing in its fully developed state.

According to this definition, software is a subset of the set of things called information. Information is "knowledge obtained from investigation, study, or instruction."² Loosely speaking, information as we are using it here is an idea or concept, or groups of these things. The notion that software is an idea or concept--something that is formed in the mind and thus easily changed--is suggested by the prefix "soft" in the word "software." In any event, the fact that software as we are using the term has a quality that makes it easy to change has significant implications with respect to SCM. This susceptibility to change, this plasticity, is the primary reason why disciplining the software development process is essential.

The first characteristic that distinguishes software from other kinds of information is that it is "structured with logical and functional properties." What this characteristic means is that software is information arranged according

to certain rules of logic in such a manner that it describes

- A specific function, or functions, to be performed
- The inputs to this (these) function(s)
- The outputs generated by this (these) function(s)

The second characteristic that distinguishes software from other kinds of information is that it is "created and maintained in various forms and representations during its life cycle." What this characteristic means is that software is information that manifests itself in one or more different ways at various stages during its evolution. This evolution, if it reaches the stage in which software is actually operating computer hardware, involves a transforming of human ideas into something that makes a piece of hardware operate. Thus, software can exist in two basic forms:

1. Non-executable form which can be further classified by the following representations:
 - a. documentation such as specifications of functions to be performed and their inputs and outputs, listings of program language statements, and flow charts that diagram the logic of the functions to be performed, and the
 - b. information embedded in machine processible carriers such as card decks, punched paper tapes, magnetic tapes, disk packs and other storage media that carry computer programs and supporting data.
2. Executable form which consists of executable (i.e., processible) sequences of information directly accessible by, and

unique to, a given suite of processing machinery.

For SCM purposes--and this point is essential to proper understanding of this paper--the two forms of software cited above are equivalent. The first form is necessary for describing the software and facilitates user communication with the processing machinery. The second form is necessary for the execution of the software on the processing machinery. The executable form of software does not exist early in the software development cycle. In fact, software embedded in machine processible carriers (the second class of non-executable representations listed above) can be thought of as a transition form in the following sense: If software is to achieve its executable form, it must first pass from its initial non-executable form (viz., documentation--the first class of non-executable representations listed above) into a form that is machine processible. We note that multiple languages may be used to describe the non-executable form of software. Examples are English prose for a design specification, a programming language such as FORTRAN or sequences of programmable calculator keystrokes for a source listing, and binary or hexadecimal languages for a derivative of source code.³

Many problems result from the potential existence of these two forms of software. At present, we wish to emphasize the following point:

BECAUSE SOFTWARE EXISTS IN TWO FORMS WHEN IT BECOMES EXECUTABLE ON PROCESSING MACHINERY, AND BECAUSE VARIOUS WAYS EXIST TO DOCUMENT THE NON-EXECUTABLE FORM, AND BECAUSE THERE ARE ALMOST ALWAYS MULTIPLE CARRIERS OF THE SOFTWARE, THERE IS A SIGNIFICANT PROBLEM IN MAINTAINING CORRESPONDENCE AND IDENTITY BETWEEN THE TWO FORMS OF SOFTWARE. MOREOVER, IT IS GENERALLY DIFFICULT TO KEEP THE VARIOUS REPRESENTATIONS OF DOCUMENTATION OF THE FIRST FORM CONSISTENT AMONG THEMSELVES.

The third characteristic that distinguishes software from other kinds of information is that it is "tailored for machine processing in its fully developed state." What this characteristic means is that when software passes from its non-executable form as documentation to its non-executable form on machine-processible carriers, it makes this passage in a special way. The information that constitutes the software is placed on these carriers in such a way that it can be accepted by hardware components of the machine that will then ultimately convert this non-executable form to the executable form. Examples of such hardware components are paper tape readers, card readers, magnetic tape and disk drives, and optical character readers.

Now that we have defined the notion of software, it is possible to define in greater detail the scope of SCM. As was maintained earlier, SCM is simply CM tailored to systems, or portions of system, that are comprised predominantly of software. More explicitly:

SCM, LIKE CM, IS DEFINED AS THE DISCIPLINE OF IDENTIFYING THE CONFIGURATION OF A SYSTEM AT DISCRETE POINTS IN TIME FOR PURPOSES OF SYSTEMATICALLY CONTROLLING CHANGES TO THIS CONFIGURATION AND MAINTAINING THE INTEGRITY AND TRACEABILITY OF THIS CONFIGURATION THROUGHOUT THE SYSTEM LIFE CYCLE.

In the following sections, we will provide some insight into the concepts embodied in this definition. In the process, it should become apparent how discipline--the discipline of SCM--can help solve traditional software development problems such as schedule delays, cost overruns, and unsatisfied system requirements.

3.0 THE SYSTEM LIFE CYCLE AND SOFTWARE CONFIGURATION MANAGEMENT

Like a biological system, a system consisting of hardware and/or software proceeds through a sequence of stages called a life cycle. This sequence begins with the system's conception (i.e., formulation of the system concept). Following this initial stage, the system concept is brought to maturity through various development stages during which the concept is transformed into hardware/software, tested and refined. With the completion of these development stages, the system has matured to the point that it can be mass-produced and deployed. After deployment, the system enters its final, but most significant stage: the operational stage. Like the life cycle of a biological system, the life cycle of a system consisting of hardware and/or software terminates when the operational stage terminates.

THE PRIMARY OBJECTIVE OF CM IS THE COST-EFFECTIVE MANAGEMENT OF A SYSTEM'S LIFE CYCLE. A concept fundamental to this management process is "baseline." A baseline is a reference point or plateau in the development of a system; a baseline is formally defined at the end of each stage in the system life cycle.

The development and production of software differs from the development and production of hardware. For example, unlike hardware, software is usually designed and built to be one-of-a-kind; the mass production of software is a copying process unlike the mass production of hardware which is a manufacturing process. Thus, when hardware mass production is undertaken, the cost of correcting flaws uncovered during development may not be large when amortized over the production units.

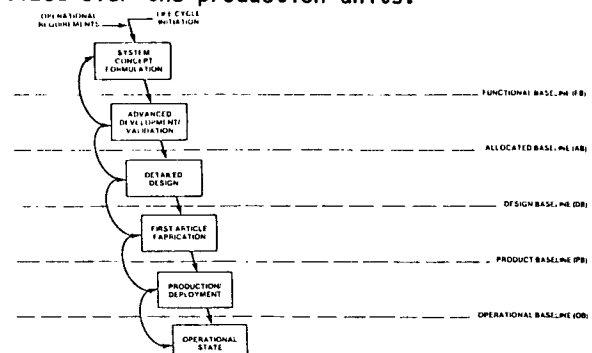


FIGURE 3-1. THE SYSTEM LIFE CYCLE AND ASSOCIATED BASELINES

Figure 3-1 depicts the six stages of the system life cycle. Each of these stages is equally applicable to hardware and software. However, these concepts have been traditionally applied only to hardware. The differences between hardware and software development and production give rise to requirements for managing the software life cycle that differ in specifics from the requirements for managing the hardware life cycle. Each of the stages of development culminates in a baseline, a point at which management has the opportunity to view the system in detail in order to examine its integrity, i.e., its adherence to and satisfaction of the operational requirements.

The Functional Baseline is typically a requirements document. It outlines, in terms that both the buyer of the system and the developer of the system can understand and agree to, exactly what the system will do. The Allocated Baseline defines precisely what functions will be performed by the hardware components of the system and what functions will be performed by the software components. This baseline may result in two documents that are used by the respective development activities in building and integrating the system. The third baseline, the Design Baseline, is well founded in the hardware world, but has been traditionally neglected by software developers. For the hardware designer, the Design Baseline includes such things as approved and controlled engineering drawings, and power and form-fit specifications. For the software designer, this baseline marks the final stage in the software life cycle before software code materializes. It should provide the software development manager a precise means for defining, controlling, and assessing the integrity of the software components of the system. The remaining baselines, the Product Baseline

and Operational Baseline represent, respectively, the milestones of the first article (prototype) development and the operational system. In these stages of the software life cycle, code is written, debugged, and optimized and then integrated and tested with the system hardware. The baselines provide the manager with the means by which he/she may trace back, through previous baselines, to ensure that the product being built is indeed the product desired.

As Figure 3-1 shows, there is, and should be, continuing feedback and interaction among development stages and their resultant baselines. Thus, any changes to the system that arise at any stage of the development process may then be fed back to previous design/specification stages and their impact assessed and promulgated to the other developing activities. This process promotes consistency across all development disciplines as well as accuracy in the supporting documentation.

System life cycle management is equivalent to baseline management.

The balance of this paper deals with the four component elements of Configuration Management as viewed in the context of software (SCM). Figure 3-2 introduces the terms that will be further explored in the following sections.

4.0 SOFTWARE CONFIGURATION IDENTIFICATION

Effective management of the development of a system requires careful definition of its baseline components; changes to these components also need to be defined since these changes, together with the baselines, specify the system evolution. A SYSTEM BASELINE IS LIKE A SNAPSHOT

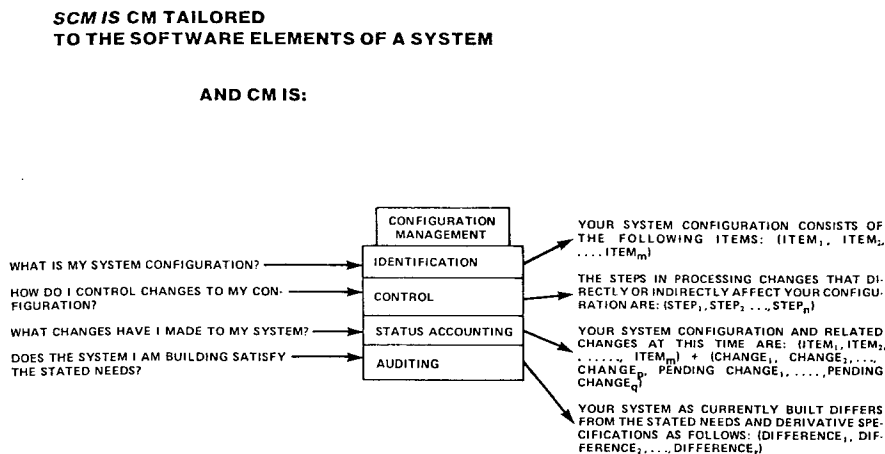


FIGURE 3-2. WHAT IS SCM?

OF THE AGGREGATE OF SYSTEM COMPONENTS AS THEY EXIST AT A GIVEN POINT IN TIME; UPDATES TO THIS BASELINE ARE LIKE FRAMES IN A MOVIE STRIP OF THE SYSTEM LIFE CYCLE. THE ROLE OF SOFTWARE CONFIGURATION IDENTIFICATION IN THE SCM PROCESS IS TO PROVIDE LABELS FOR THE CONTENTS OF THESE SNAPSHOTS AND THE MOVIE STRIP. This labeling mechanism provides the apparatus for accomplishing the other three SCM functions described in the following three sections.

Five baselines are established during a software system's life cycle. These baselines were indicated in Figure 3-1. Each baseline controls subsequent system development. At the time it is first established, a software baseline embodies the actual software in its most recent state. When changes are made to the most recently established baseline, then, from the viewpoint of the software configuration manager, this baseline and these changes embody the actual software in its most recent state (although, from the viewpoint of the software developer, the actual software may be in a more advanced state).

The most elementary entity in the Software Configuration Identification labeling mechanism is the Software Configuration Item (SCI). Viewed from an SCM perspective, a software baseline appears as a set of SCI's. The SCI's within a baseline are related to one another via a tree-like hierarchy. As the software system evolves through its life cycle, the number of branches in this hierarchy generally increases; the first baseline (i.e., the Functional Baseline) may consist of as little as one SCI. The lowest level SCI's in the tree hierarchy may still be under development and not yet be under SCM control. These entities are termed Design Objects (DO's). The relationship between SCI's and DO's is shown in Figure 4-1.⁴

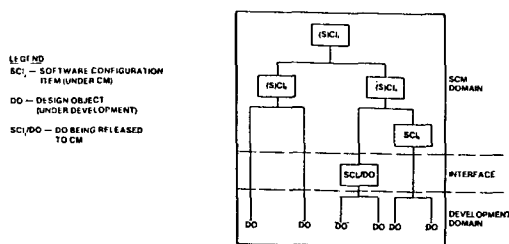


FIGURE 4-1. THE DEVELOPMENT/SCI INTERFACE

A baseline can be characterized by two labels. One label identifies the baseline itself (i.e., this label identifies the baseline to be one of the five depicted in Figure 3-1). The second label identifies an update to a particular baseline. An update to a baseline represents a baseline plus a set of changes that have been incorporated into it; a change is defined as an addition, deletion or modification of requirements (such as system, operational, or contractual).

Software can be represented in a number of ways. The most prevalent of these representations are

- (1) Specification documentation (e.g., functional, conceptual design, programming design)
- (2) Source language carried on machine-readable media
- (3) Source language hard copy listing
- (4) Object code carried on machine-readable media
- (5) Executable equivalent of (4) above

Each baseline and each member in the associated family of updates will exist in one or more of the above representations. Figure 3-1 implies that the first three baselines in a software system's life cycle (i.e., the Functional Baseline, Allocated Baseline and Design Baseline) exist only in the first representation listed above. The remaining two baselines (i.e., the Product Baseline and Operational Baseline) generally exist in all five representations. The Software Configuration Identification mechanism accounts for the multiple representations of these two baselines.

Figure 4-2 depicts the Software Configuration Identification process. The movie strip and snapshot analogies of the software system life cycle have been utilized in the figure. The figure is intended to indicate that each baseline and associated updates portray the entire history of the software system development from the time that baseline first appears. Each succeeding baseline and associated updates portray the system development at a greater level of detail than the preceding baseline and associated updates.⁵

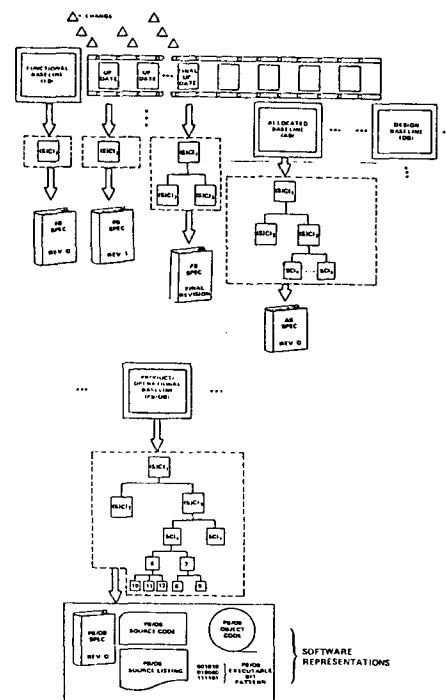


FIGURE 4-2. SOFTWARE CONFIGURATION IDENTIFICATION

5.0 SOFTWARE CONFIGURATION CONTROL

Figure 4-2 indicated that the evolution of a software system is, in the language of SCM, the development of baselines and the incorporation of a series of changes into the baselines. In addition to these changes that explicitly affect existing baselines, there are changes that occur during early stages of the system life cycle that may affect baselines that do not yet exist. For example, some time before software coding begins (i.e., some time prior to the establishment of the Design Baseline (cf. Figure 3-1)), a contract may be modified to include a software warranty provision such as: "System down time due to software failures shall not exceed thirty (30) minutes per day." This warranty provision will, in general, affect subsequent baselines (i.e., the Product Baseline and the Operational Baseline) but in a manner that cannot be explicitly determined a priori. THE ROLE OF SOFTWARE CONFIGURATION CONTROL IS TO PROVIDE THE ADMINISTRATIVE MECHANISM FOR PRECIPITATING, PREPARING, EVALUATING AND APPROVING/DISAPPROVING ALL CHANGE PROPOSALS THROUGHOUT THE SYSTEM LIFE CYCLE. THAT IS, SOFTWARE CONFIGURATION CONTROL IS CHANGE PROPOSAL PROCESSING.

Section 4 emphasized that software for configuration management purposes is a collection of Software Configuration Items (SCI's) that are related to one another in a well-defined way. In the Functional Baseline, the Allocated Baseline, and the Design Baseline and their associated updates, SCI's are specification documentation (i.e., one or more volumes of text for each baseline or associated update); in the Product Baseline and the Operational Baseline and their associated updates, each SCI may be represented in as many as five distinct ways (cf. Figure 4-2). Software configuration control focuses on the management of changes to SCI's (existing or to be determined) in all their representations.

Software configuration control involves two basic ingredients:

- (1) Documentation (e.g., administrative forms and supporting technical and administrative material) for formally precipitating and defining a proposed change to a software system
- (2) An organizational body for formally evaluating and approving/disapproving a proposed change to a software system

A principal component of the first ingredient above is called an Engineering Change Proposal (ECP) and contains information such as: a description of the proposed change, identification of the originating organization, rationale for the proposed change, identification of affected baselines and SCI's (if appropriate), and specification of cost and schedule impacts. The second ingredient above is called a Configuration Control Board (CCB). The CCB is typically comprised of a voting body representing organizational units that have a vested interest in the proposed change.

Causes that precipitate change proposals affecting baselines include

- (1) Software Deficiencies: An existing software baseline may be inadequate or incorrect by design, implementation, assumption or for other reasons.
- (2) Hardware Changes: Problems with hardware components and interfaces among hardware subsystems may yield to solution only through software modifications.
- (3) New Operational Requirements: The ground rules for the software system's operation may be modified (e.g., a reduction or enhancement in the scope of a performance mandate).
- (4) Economic Savings: Means for effecting cost savings may be determined, or lower development and/or operating costs may be decreed that require software modifications.
- (5) Schedule Accommodation: Acceleration of a system implementation schedule (e.g., for hardware, software and/or facilities) or inability to meet a pre-established system implementation schedule may lead to software modifications.

Figure 5-1 depicts the Software Configuration Control process. This figure also depicts the relationship of this administrative process to the actual implementation of the change. Change implementation is not an SCM function; however, the monitoring of the change implementation process is an SCM function (cf. Section 6). Figure 5-1 also emphasizes that the analysis that may be required to prepare an ECP is also outside of the SCM purview.

6.0 SOFTWARE CONFIGURATION STATUS ACCOUNTING

The discussion of Software Configuration Control in the preceding section suggested that changes to a software system are not made on a real-time basis. A decision to make a change (i.e., approval of an Engineering Change Proposal by the Configuration Control Board) is generally followed by a time delay before the change is actually effected. For example, even for changes not involving a modification to a baseline (such as incorporation of a software warranty provision into a contract--cf. Figure 5-1), administrative machinations may consume days (particularly in a large corporation or Government agency) before the change is implemented. Furthermore, as Figure 4-2 suggests, changes to baselines generally occur over a protracted period of time before they are incorporated into baselines as updates. A mechanism is therefore needed for maintaining a record of how the system evolved and where the system is at any point in time relative to what appears in published baseline documentation and written agreements. Software Configuration Status Accounting provides this mechanism.

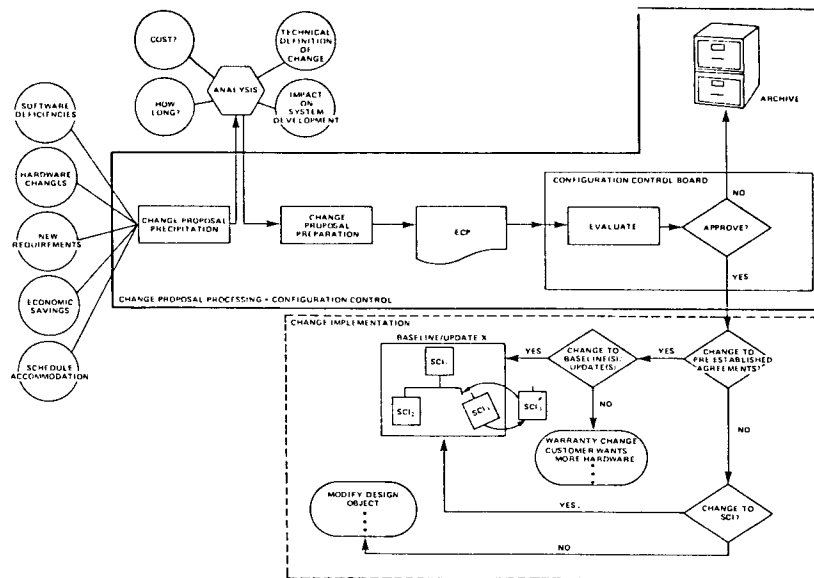


FIGURE 5-1. SOFTWARE CONFIGURATION CONTROL

SOFTWARE CONFIGURATION STATUS ACCOUNTING IS THE ADMINISTRATIVE TRACKING AND REPORTING OF ALL SOFTWARE ITEMS FORMALLY IDENTIFIED AND CONTROLLED ACCORDING TO THE METHODOLOGY DISCUSSED IN SECTIONS 4 AND 5. IT ALSO INVOLVES THE MAINTENANCE OF RECORDS TO SUPPORT SOFTWARE CONFIGURATION AUDITING TO BE DISCUSSED IN SECTION 7. SOFTWARE CONFIGURATION STATUS ACCOUNTING IS THUS THE MEANS BY WHICH THE ACTIVITY ASSOCIATED WITH THE OTHER THREE SCM FUNCTIONS IS RECORDED; IT THEREFORE PROVIDES THE MEANS BY WHICH THE HISTORY OF THE SOFTWARE SYSTEM LIFE CYCLE CAN BE TRACED.

Although administrative in nature, Software Configuration Status Accounting is a function that increases in complexity as the system life cycle progresses. This complexity becomes particularly acute with the inception of the Product Baseline and the Operational Baseline because of the multiple software representations that emerge with these baselines (cf. Figure 4-2). This complexity generally results in large amounts of data to be recorded and reported. In particular, the scope of Software Configuration Status Accounting encompasses the following recording and reporting activities:

- (1) The time at which each representation of a baseline and update came into being
- (2) The time at which each Software Configuration Item (SCI) (cf. Section 4) came into being
- (3) Descriptive information about each SCI
- (4) Engineering Change Proposal (ECP) (cf. Section 5) status (e.g., approved, disapproved, awaiting Configuration Control Board action)

- (5) Descriptive information about each ECP
- (6) Change status (i.e., status of a change as defined in an approved ECP) (cf. Section 5)
- (7) Descriptive information about each change
- (8) Status of technical and administrative documentation associated with a baseline or update (such as a plan prescribing tests to be performed on a baseline for updating purposes)
- (9) Deficiencies in a to-be-established baseline uncovered during a configuration audit (cf. Section 7).

Software Configuration Status Accounting, because of its large data input and output requirements, is generally supported in part by automated means. Data are collected and organized in a manner that facilitates inputting to a computer; reports giving the status of entities discussed in the list above are compiled and generated by the computer.

Figure 6-1 depicts the Software Configuration Status Accounting process and its relationship to the other three SCM functions.

7.0 SOFTWARE CONFIGURATION AUDITING

Software Configuration Status Accounting discussed in Section 6 provides the data source for determining the current state of the software system. Software Configuration Auditing provides the mechanism for determining the degree to which the current state of the software system mirrors the software system pictured in

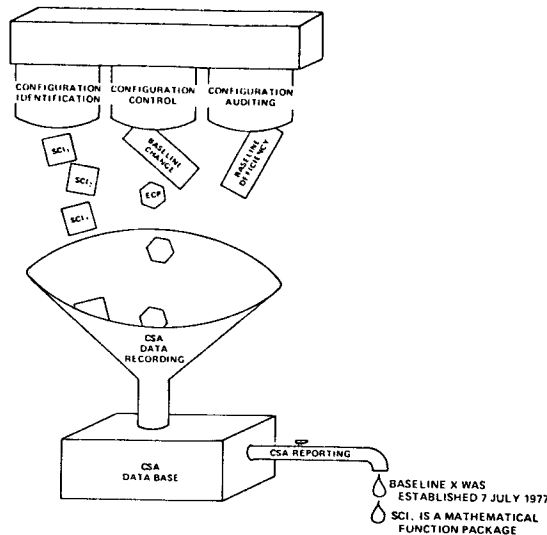


FIGURE 6-1. SOFTWARE CONFIGURATION STATUS ACCOUNTING (CSA)

documented baselines and requirements documentation. It also provides the mechanism for formally establishing a baseline. A baseline in its formative stages (e.g., a draft specification document that appears prior to the existence of the Functional Baseline) is referred to as a "to-be-established baseline"; the final state of the auditing process conducted on a to-be-established baseline is the sanctioning of the baseline. Analogous remarks apply to baseline updates.

SOFTWARE CONFIGURATION AUDITING SERVES TWO PURPOSES:

- (1) CONFIGURATION VERIFICATION
- (2) CONFIGURATION VALIDATION

VERIFICATION INSURES THAT WHAT IS INTENDED FOR EACH SOFTWARE CONFIGURATION ITEM (SCI) AS SPECIFIED IN ONE BASELINE OR UPDATE IS ACTUALLY ACHIEVED IN A SUCCEEDING BASELINE OR UPDATE; VALIDATION ENSURES THAT THE SCI CONFIGURATION SOLVES THE RIGHT PROBLEM (I.E., CUSTOMER REQUIREMENTS ARE FULFILLED).

Software Configuration Auditing is applied to each of the baselines (and corresponding updates) identified in Figure 3-1, in its to-be-established state. An auditing process common to all baselines is the determination that an SCI structure exists and that its contents are based on all available information. Some particulars of the auditing process vary with the baseline being considered. For example:

- (1) For the Functional Baseline, the Software Configuration Auditing focuses on establishing that there is a clear trace between overall system requirements and software requirements.⁶

- (2) For the Allocated Baseline, Software Configuration Auditing focuses on establishing that the software functions defined in the Functional Baseline have been definitized into implementable models and algorithms.⁷

- (3) For the Design Baseline, Software Configuration Auditing focuses on establishing that the models and algorithms definitized in the Allocated Baseline have been detailed sufficiently to permit unambiguous software coding.

- (4) For the Product Baseline, Software Configuration Auditing focuses on establishing that the executable representation of the SCI's performs adequately (i.e., satisfies overall system operational needs) in a test environment.

- (5) For the Operational Baseline, Software Configuration Auditing focuses on establishing that the executable representation of the SCI's performs adequately in an operational environment.

Figure 7-1 depicts the Software Configuration Auditing process.

8.0 SUMMARY

We have highlighted the basic elements of the management discipline called Software Configuration Management (SCM). We have discussed the concept of software and have indicated how software's susceptibility to change gives rise to a need for disciplining software configuration development if software product integrity is to be attained. This discipline is achieved

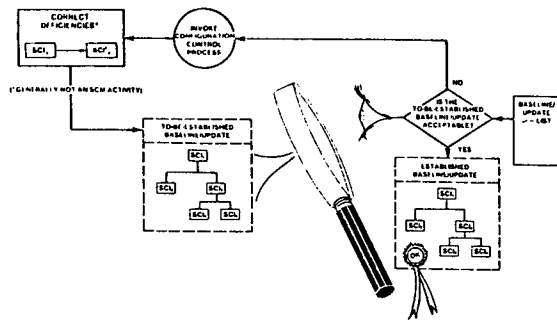


FIGURE 7-1. SOFTWARE CONFIGURATION AUDITING

through a process of baseline management. Baseline management consists of the identification, control, status accounting, and auditing of a collection of software configuration items that define a software configuration. This management process permits the explicit monitoring of software evolution throughout all stages of the system life cycle.

9.0 FOOTNOTES

1. The term machine as used in this context refers to hardware that hosts software and specifically excludes biological machines such as people.
2. Webster's New Collegiate Dictionary, G&C Merriam Company, Springfield, Massachusetts, 1977, p. 592 (the definition given above is the second one listed).
3. This derivative language is generally called object code in that it represents the object of a translation process from source language (which is more natural for humans) to a binary-based language (which most types of computer processing machines find more palatable).
4. In Figure 4-1, some SCI's are written as (S)CI. The parentheses around the S are meant to suggest that early in the system life cycle a configuration item may not be associated with software functions.
5. It should be noted that, because of the continuing feedback and interaction among the development stages shown in Figure 3-1, more than one baseline may be updated at the same time. In general, because software is susceptible to change, a given baseline often does not experience a "final update" until development of the software ceases (which may occur during any of the stages shown in Figure 3-1).
6. The Functional Baseline describes a system in its formative stage. In this stage, the hardware and software roles may not be delineated (at least in the initial version of the Functional Base-

line). Consequently, the concept of Functional Baseline is often something that is of interest to the system configuration manager as opposed to the software configuration manager (who may not be an organizational entity during the formative stage of a system's life cycle).

7. The allocated baseline can be viewed as a more detailed functional specification of what the system is supposed to do after it has been decided what roles the hardware and software components are to play.

10.0 BIBLIOGRAPHY

- a. Bersoff, E. H., V. D. Henderson, and S. G. Siegel, Principles of Software Configuration Management, Prentice-Hall, Inc., Englewood Cliffs, New Jersey, to be published.
- b. Czerwinski, F. L., and T. T. Samaras, Fundamentals of Configuration Management, John Wiley & Sons, Inc., New York, New York, 1971 (esp. Chapter 21, "Configuration Management of Operational Computer Programs").
- c. Metzger, P. W., Managing a Programming Project, Prentice-Hall, Inc., Englewood Cliffs, New Jersey, 1973 (configuration management is discussed on pp. 136 ff.).
- d. Weiss, D. M., "The MUDD Report: A Case Study of Navy Software Development Practices," Naval Research Laboratory Report 7909, 21 May 1975.
- e. Department of Defense Joint Service/Agency Regulation, "Configuration Management," July 1974.
- f. Military Standard MIL-STD-480, "Configuration Control--Engineering Changes, Deviations, and Waivers," 30 October 1968.
- g. Military Standard MIL-STD-481A, "Configuration Control--Engineering Changes, Deviations, and Waivers (short form)," 18 October 1972.
- h. Military Standard MIL-STD-482, "Configuration Status Accounting Data elements and Related Features," 31 December 1970.
- i. Military Standard MIL-STD-483 (USAF), "Configuration Management Practices for Systems, Equipment, Munitions, and Computer Programs," 31 December 1970.
- j. Military Standard MIL-STD-1456, "Contractor Configuration Management Plans," 25 January 1972.